

PhreshPhish Reproduction Study: Critical Evaluation of Phishing Detection Benchmarks

Student: Aya Khawaled (ID: 214742496)

Course: Data Science in Cyber - Final Project

Instructor: Dr. Uri Itai

Source: Dalton et al., PhreshPhish: A Real-World, High-Quality, Large-Scale Phishing Website Dataset and Benchmark (arXiv:2507.10854)

Repository: Data-Science-in-Cyber-Final-Project

Executive Summary

This project reproduces a simplified phishing-detection pipeline inspired by PhreshPhish and critically evaluates the authors' central claim: many published results are misleading because benchmarks use unrealistic phishing prevalence. We trained Logistic Regression and Random Forest on URL/HTML features extracted from a 5,000-train / 1,000-test subset of the PhreshPhish Hugging Face dataset.

On the official-style test split (~45% phishing), Random Forest achieved 93% precision and 0.905 F1. On a simulated 1% phishing deployment benchmark (test phishing positives + pooled benign rows), precision fell to 51% (bootstrap 95% CI: 0.37-0.63) while ROC-AUC remained ~0.99. A prevalence sweep (1%-45%) showed precision and F1 rising monotonically as base rate increased.

Conclusion: Our experiments support the authors' argument that high-prevalence evaluation overstates practical usefulness. We did not test leakage claims. The reproduction uses a subset and simpler features than the full paper baselines.

1. Summary of the Source

1.1 Problem

Phishing websites steal credentials and financial data at scale. Machine learning detectors are widely studied, but Dalton et al. argue that many prior datasets and benchmarks are low quality, suffer from train-test leakage, and use unrealistic class balance (~50% phishing in lab settings vs. ~1% or less in the wild). Inflated metrics from such setups mislead researchers and practitioners.

1.2 Why it matters

False negatives (missed phishing) expose users to fraud. False positives (blocking legitimate sites) erode trust and create support costs. Evaluation must reflect deployment prevalence, not only ranking accuracy on balanced test sets.

1.3 Proposed solution

PhreshPhish releases a large dataset of URL + HTML pairs with train/test splits and benchmark protocols emphasizing realistic base rates. The authors provide baselines (including n-gram and stronger models) and argue for rigorous, leakage-aware evaluation.

1.4 Dataset

- Name: PhreshPhish
- Access: Hugging Face - [phreshphish/phreshphish](#)
- Content: Phishing and benign webpage URL + HTML
- Our subset: 5,000 train + 1,000 test rows (streamed; HTML capped at 80,000 chars per row; disk cache in data/)
- Train prevalence: 43.78% phishing (2,189 / 5,000)
- Test prevalence: ~45.3% phishing

1.5 Methodology (paper vs. our reproduction)

Aspect	PhreshPhish (paper)	Our reproduction
Features	Rich baselines (e.g. n-grams, LLM)	19 hand-crafted URL/HTML features (src/features.py)
Models	Multiple strong baselines	Logistic Regression, Random Forest
Scale	Full dataset (~36 GB)	5k/1k subset
Low-rate eval	Official benchmark	Simulated 1% slice + prevalence sweep

2. Critical Evaluation

2.1 Main claims by the authors

Prior phishing ML work often uses low-quality data and leakage, inflating results.

Unrealistic base rates make models appear better than they would deploy.

PhreshPhish provides a rigorous benchmark with realistic prevalence settings.

2.2 What we tested

We focused on claim #2 (base rate), because our pipeline and data subset allow a controlled comparison: same models and features, two evaluation regimes (high vs. ~1% prevalence).

We did not empirically audit leakage (duplicate URLs across splits, temporal overlap, etc.). Our train subset had zero duplicate URLs in the sampled 5,000 rows, but this is not a full leakage study.

2.3 Is the evaluation methodology appropriate?

For the paper's goal: Yes - separating "easy" high-prevalence tests from rare-phishing benchmarks is appropriate for cybersecurity.

For our reproduction: Partially. We:

- Used official train/test splits from Hugging Face
- Held out test positives for the 1% benchmark
- Pooled train benign rows to enlarge the negative pool (positives always from test)

This is a reasonable simulation but not identical to the paper's official low-rate benchmark protocol.

2.4 Weaknesses and limitations

- Subset only - results may not match full-scale PhreshPhish numbers.
- Simple features - paper baselines are stronger; our models are teaching/reproduction baselines.
- 1% benchmark size - only 34 phishing positives in the expanded pool; bootstrap CIs are wide.
- Leakage untested - cannot confirm or refute claim #1.
- Threshold fixed at 0.5 - deployment would tune threshold for cost asymmetry.

2.5 Are conclusions justified?

Supported by our evidence: High-prevalence test metrics overstate precision/F1 relative to a ~1% deployment-like setting. Random Forest precision dropped from 0.93 to 0.51 (45% relative decrease) with stable multi-seed behavior (precision 0.496 +/- 0.017).

Not contradicted, but not fully replicated: Leakage critique and full benchmark numbers from the paper.

3. Feature Engineering Analysis

3.1 Overview

Features were engineered in `src/features.py` from raw URL and HTML strings - no categorical encoding was needed (all numeric).

URL features (11): URL/host/path length, dots, hyphens, digits, subdomains, HTTPS flag, @ symbol, IP-host flag, suspicious-token count (login, verify, bank, etc.).

HTML features (8): length, forms, inputs, password-input count, password-field flag, scripts, anchor

links, iframes.

Total: 19 numeric features per row.

3.2 Transformations applied

Step	Method	Why	Effect
Missing HTML/URL	fillna("") before extraction	Avoid parse errors	No missing values in feature matrix
Logistic Regression input	StandardScaler in Pipeline	LR sensitive to scale	Stable coefficients and convergence
Random Forest input	No scaling	Tree splits are scale-invariant	-
Redundancy removal	Drop pairs with $ \text{Pearson} > 0.9$	Reduce multicollinearity	None dropped in our subset
Class imbalance	class_weight='balanced'	~44% phishing in train	Reduces bias toward majority class

No log/Box-Cox transforms were applied; URL lengths are right-skewed but tree models and scaled LR handled them adequately for this baseline study.

3.3 Redundancy analysis

We computed Pearson and Spearman correlation heatmaps on training features. Pearson assumes linearity; Spearman is more robust to skew and outliers (important for url_length). Kendall tau for (url_length, num_dots) was 0.279 ($p < 0.001$), indicating a moderate monotonic relationship - longer URLs tend to have more dots (subdomains/path), which is intuitive for phishing clones.

No feature pairs exceeded $|r| > 0.9$, so no columns were removed.

3.4 Cybersecurity meaning

- Suspicious URL tokens capture social-engineering paths (e.g. login, verify).
- Password fields / forms in HTML reflect credential-harvesting pages.
- HTTPS flag alone is weak (many phishing sites use TLS); combined features matter.

3.5 Features that could improve performance

- DOM-derived features with a real browser (rendered text, favicon, certificate)
- N-gram TF-IDF on HTML text (as in paper baselines)
- Domain age, WHOIS, and hosting reputation
- Temporal features if crawl date is used carefully to avoid leakage

4. Reproducibility Analysis

4.1 Code execution

The notebook `notebooks/phreshphish_analysis.ipynb` runs end-to-end with `requirements.txt`.
Supporting modules: `src/features.py`, `src/data_cache.py`.

4.2 Dependencies and data

- Hugging Face datasets for streaming
- First run downloads a subset; later runs load from `data/*.parquet` cache
- `FORCE_REBUILD_CACHE = True` re-streams from Hugging Face

4.3 Hidden steps

- HTML truncated to 80,000 characters per row (memory safety) - documented in config
- Column name resolution for Hugging Face schema variants - in notebook

4.4 Overall reproducibility

Good for a subset reproduction with fixed `RANDOM_STATE = 42`. Full 36 GB replication requires more disk and time than we used. The PhreshPhish GitHub repo and Hugging Face dataset provide sufficient public artifacts to reproduce at scale.

5. Experimental Results

5.1 Experiments performed

- EDA on 5,000 training rows (distributions, imbalance, correlations, outliers)
- Train Logistic Regression and Random Forest on reduced feature matrix
- Evaluate on test subset (~45% phishing, $n = 1,000$)
- Evaluate on simulated 1% benchmark (34 pos / 3,392 total, expanded benign pool)
- Prevalence sweep at 1%, 5%, 10%, 25%, 45%
- Bootstrap (500) and multi-seed (10) stability on 1% benchmark
- Error analysis on Random Forest test predictions

Modifications we introduced (vs. the paper): a reduced 5k/1k streamed subset; 19 hand-crafted features instead of the paper's heavier baselines; a self-constructed 1% prevalence benchmark (test-only positives, benign pooled from train+test); a prevalence sweep; and bootstrap + multi-seed robustness checks not present in the original.

5.2 Models

- Logistic Regression: StandardScaler + LogisticRegression(class_weight='balanced', max_iter=1000)
- Random Forest: RandomForestClassifier(n_estimators=200, class_weight='balanced')

5.3 Metrics (classification)

We report Accuracy, Precision, Recall, F1, MCC, ROC-AUC and confusion matrices.

Cybersecurity interpretation:

- Precision: Of flagged pages, how many are truly phishing? Low precision means many legitimate sites blocked (FP).
- Recall: Of true phishing, how many caught? Low recall means attacks reach users (FN).
- ROC-AUC: Ranking quality across thresholds; can stay high even when precision at default threshold is poor.

5.4 Results - high-prevalence test (~45% phishing)

Model	Accuracy	Precision	Recall	F1	MCC	ROC-AUC
Logistic Regression	0.860	0.830	0.870	0.849	0.719	0.937
Random Forest	0.916	0.928	0.883	0.905	0.831	0.974

Best model: Random Forest (highest F1).

5.5 Results - simulated 1% phishing (expanded pool)

Model	Precision	Recall	F1
Logistic Regression	0.064	0.912	0.120
Random Forest	0.508	0.941	0.660

Random Forest bootstrap at 1%: precision 0.503 [0.374, 0.629]; multi-seed precision 0.496 +/- 0.017.

5.6 Prevalence sweep (both models)

We swept the simulated phishing prevalence for both models to confirm the base-rate effect is not specific to Random Forest.

Model	Prevalence	Precision	Recall	F1	ROC-AUC
Logistic Regression	1%	0.064	0.912	0.120	0.958
Logistic Regression	5%	0.253	0.864	0.391	0.935
Logistic Regression	10%	0.418	0.871	0.565	0.937
Logistic Regression	25%	0.465	0.870	0.606	0.938
Logistic Regression	45%	0.465	0.870	0.606	0.938
Random Forest	1%	0.508	0.941	0.660	0.998
Random Forest	5%	0.834	0.881	0.857	0.993
Random Forest	10%	0.914	0.882	0.898	0.994
Random Forest	25%	0.928	0.883	0.905	0.993

Model	Prevalence	Precision	Recall	F1	ROC-AUC
Random Forest	45%	0.928	0.883	0.905	0.993

For both models, precision and F1 rise as prevalence increases while recall and ROC-AUC stay roughly flat - direct evidence that the same model looks "better" on imbalanced-easy tests. The effect is even more severe for the weaker Logistic Regression (precision 0.06 at 1% vs 0.47 at 45%), and Random Forest dominates at every prevalence. This shows the base-rate distortion is a property of the evaluation prevalence, not of one specific model.

5.7 Error analysis (Random Forest, test set)

- False positives: 31 (benign flagged as phishing)
- False negatives: 53 (phishing missed)

Trade-off: At high prevalence, missing 53 phishing pages is the dominant operational risk (FN). At 1% deployment with threshold 0.5, the dominant issue shifts to false alarms (low precision) - blocking legitimate sites.

6. Conclusions

6.1 Key findings

Random Forest performs well on the standard test split (F1 0.91, ROC-AUC 0.97).

The same model at ~1% phishing drops to ~51% precision - flagged pages are wrong about half the time at the default threshold.

ROC-AUC stays high (~0.99 at 1%), so the model ranks phishing above benign, but threshold and prevalence determine operational precision.

The precision/F1 drop at low prevalence appears for both models (Logistic Regression and Random Forest), so PhreshPhish's warning about unrealistic base rates is supported by our reproduction and is not an artifact of a single model.

6.2 Lessons learned

- Report multiple evaluation regimes, not only balanced or high-prevalence test sets.
- Precision at deployment prevalence matters as much as recall for user trust.
- Disk caching and streaming are necessary for large HTML datasets on consumer hardware.

6.3 Strengths of PhreshPhish

- Large, real-world URL+HTML pairs
- Explicit discussion of leakage and base-rate problems
- Public dataset and benchmarks

6.4 Weaknesses / our study limits

- Full benchmark replication needs more compute and stronger features
- Our 1% slice has few positives (statistical noise)
- Leakage claim not independently verified

6.5 Future improvements

- Larger sample (10k+ test) and official low-rate protocol
- Threshold tuning on validation data for target precision
- N-gram / transformer baselines from the paper
- Leakage audit (URL overlap, temporal split analysis)

6.6 Recommendation

Yes, for phishing detection research - PhreshPhish is a valuable reference for how to evaluate detectors, not only for raw accuracy on a single split. Practitioners should always ask: "At what phishing prevalence were these metrics measured?"

Summing It Up

Item	Summary
Problem	Misleading ML metrics for phishing due to poor data and unrealistic prevalence
Source	PhreshPhish (Dalton et al., 2025) - arXiv:2507.10854
Dataset	PhreshPhish subset via Hugging Face (5k train / 1k test)
Methodology	URL/HTML features, Logistic Regression vs Random Forest, dual evaluation (45% vs 1%)
Main finding	Precision 0.93 (test) vs 0.51 (1% benchmark) - same model, different story
Claims supported?	Base-rate claim: yes. Leakage claim: not tested.
Key insight	Good ROC-AUC does not mean good deployment precision when phishing is rare
Use on similar problems?	Recommended as evaluation framework; reproduce with realistic prevalence

References

Dalton et al., PhreshPhish: A Real-World, High-Quality, Large-Scale Phishing Website Dataset and Benchmark, arXiv:2507.10854, 2025.

PhreshPhish GitHub: <https://github.com/PhreshPhish/phreshphish>

Dataset: <https://huggingface.co/datasets/phreshphish/phreshphish>

Notebook and code: [notebooks/phreshphish_analysis.ipynb](#), [src/features.py](#), [src/data_cache.py](#)